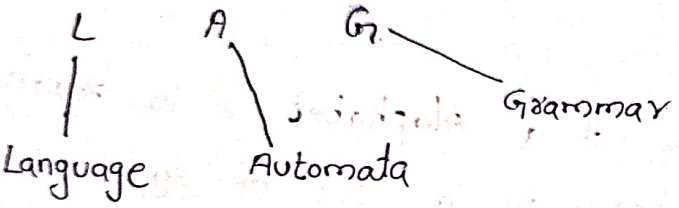
Theory of Computation: To study the computer, first develop the mathematical model (or) abstract model. This is called theory of computation.

The theory of computation is based on 3 pillers.



Language:

(i) symbol: Building block (or) the smallest unit of the language.

Ex: $\{a, b, c, 0, \ldots\}$

(ii) Alphabet: An alphabet is a finite non-empty set of symbols, denoted by Σ.

Ex: lowercase letters $\Sigma = \{a, b, \ldots\}$

binary numbers $\Sigma = \{0, 1\}$

numeric numbers $\Sigma = \{0, 1, \ldots 9\}$

(iii) String: A string (or) word is a finite sequence of symbols choose from Σ. Empty string is denoted by ε. string (or) word can be denoted by ω.

Ex: $\{aa, bb\}$, $\{abba, aaaa\}$, $\{01, 10, 100, 101\}$

(3) Length of a string: number of meaningful symbols present in the strings. If s is string then its length is denoted by $|s|$

Ex: If $x = abbaab$ then $|x| = 6$

If $|x| = 0$ then it is called empty string.

→ Power of a string: If Σ is an alphabet then Σ^K is set of all strings of length K.

Ex: $\Sigma = \{0,1\}$ then $\Sigma^1 = \{0,1\}$ $\Sigma^2 = \{00,01,10,11\}$

$\Sigma^3 = \{001, 010, \text{------}\}$

→ Kleen closure:

→ A kleen closure of a alaphabet Σ is represented by Σ^*

→ Σ^* is a operator on a set of symbols (or) strings on Σ that gives infinite set of all possible strings of all possible lengths over Σ including ε.

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \text{----------}$$

if $\Sigma = \{0,1\}$

$\Sigma^* = \{\varepsilon, 0, 1, 01, 10, 11, 00, 110, 111, 000, \text{------}\}$

↓ empty set

→ Kleen closure plus (or) positive closure:

→ Kleen closure plus is represented by Σ^+

→ Σ^+ is the set of all possible strings of all possible lengths over Σ excluding ε

$$\Sigma^+ = \Sigma^* - \varepsilon$$

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \text{----------}$$

Ex: $\Sigma = \{0,1\}$

$\Sigma^+ = \{0, 1, 10, 00, 11, \text{------}\}$

Language: A language is a subset of Σ^* for some alphabet Σ it can be finite (or) infinite

Ex: $\Sigma = \{a, b\}$

L be the language of all strings of equal no. of a's and b's.

$L = \{\varepsilon, ab, ba, aabb, bbaa, abab, \ldots\ldots\}$

ϕ denotes empty Language.

Types of Languages:

Language can be divided in to 2 Types.

① Finite Language (Finite no. of strings)

② Infinite Language (Infinite no. of strings)

Ex: L_1 = Length 2.

$\Sigma = \{a, b\}$

$L_1 = \{aa, ab, ba, bb\}$

L_2 = at least 1a.

$\Sigma = \{a, b\}$

$L_2 = \{a, aa, aaa, abb, aba, bab, \ldots\ldots\ldots\}$

Suppose we have one string $W = \{abba\}$. now check the string is related to language (or) not.

For testing of strings

→ Finite and Infinite strings are stored in memory.

→ now check the string with the Finite strings. For finite strings it is possible for checking the particular string.

→ next the string is check with the infinite strings. For infinite strings it is difficult for checking the string.

For infinite strings, we can use automata.

Automata is a mathamatical model (or) machine

Automata mainly used to check the string is belongs to language (or) not

Grammar:

→ Grammar is same as in natural language like english.

→ Grammar checks

(i) weather the sentence is related to Language (or) not.

(ii) Grammar means standard way of representing the language.

→ Grammar mainly used for weather the string is part of Language (or) not.

→ A Grammar G is defined as 4-tuple (or) quardraple.

$$G = \{V, T, P, S\}$$

V = Variable = non-Terminal symbols

T = Terminal symbols

P = Production Rule

S = Start Symbol

Terminal symbol : Terminal symbols are the components of the sentences that are generated using grammar and are denoted using lower case letters like a,b,c etc.

Non-Terminal symbols: Non-Terminal symbols are also called Auxiliary symbols (or) variables.

→ Non-Terminal symbols take part in the generation of the sentence, but not the components of the sentence.

→ They are represented by using capital letters like A, B, C etc.

Production Rule : Production rule (P) : $\alpha \to \beta$

→ Based on the given production rule we can generate strings, lang-

$\alpha \to$ Left production

$\beta \to$ Right Production

Language generated from a Grammar:

Derivation from a Grammar:

→ The set of all strings that can be derived from a grammar is said to be the Language derived from that given grammar.

① $G_2 = (\{S, A, B\}, \{a, b\}, S, \{S \to AB, A \to a, B \to b\})$

$S \to AB$

$\to ab \quad (A \to a, \ B \to b)$

$L(G_2) = \{ab\}$

② $G_3 = (\{S, A, B\}, \{a, b\}, S, \{S \to AB, A \to aA/a, B \to bB/b\})$

$S \to AB$

$S \to ab \quad (A \to a, \ B \to b)$

$S \to AB$

$S \to aAbB$

$\to aabb$

$S \to a^2b^2$

$S \to AB$

$S \to aAb$

$S \to aab$

$S \to a^2b$

$S \to AB$

$S \to abB$

$S \to abb$

$S \to ab^2$

$L(G_3) = \{ab, a^2b^2, a^2b, ab^2, \ldots\}$

$= \{a^m b^n / m \geq 0 \text{ and } n \geq 0\}$

③ Let $G_1 = (\{S, C\}, \{a, b\}, \{S \to aca, C \to aca, C \to b\}, S\}$ Find $L(G_1)$

$S \to aca$

$S \to aaacaaa$

$S \to aabaa$

$S \to aacaa$

$S \to aaacaaa$

$S \to aaabaaa$

$L(G_1) = \{aba, aabaa, aaabaaa, \ldots\}$

$L(G_1) = \{aba, a^2ba^2, a^3ba^3, \ldots\}$

$L(G_1) = \{a^n b a^n \text{ where } n \geq 1\}$

(4) Consider the Grammar $G1 = (\{S, A\}, \{a, b\}, S, \{S \to aAb, aA \to aaAb, A \to \varepsilon\})$

$S \to aAb$

$S \to ab \quad (A \to \varepsilon)$

$\to ab$

$S \to aAb$

$\to aaAbb \quad (aA \to aaAbb)$

$\to aabb \quad (A \to \varepsilon)$

$\to a^2b^2$

$S \to aAb$

$S \to aaAbb \quad (aA \to aaAb)$

$\to aaaAbbb \quad (aA \to aaAb)$

$\to aaabbb \quad (A \to \varepsilon)$

$\to a^3b^3$

$L(G) = \{ab, a^2b^2, \ldots\ldots a^n b^n \ldots\ldots\}$

$L(G) = \{a^n b^n / n \geq 1\}$

Chomsky Hierarchy of language (or) Types of Languages

→ There are 4 types of Grammars

(i) Type 0 Grammar

(ii) Type 1 Grammar

(iii) Type 2 Grammar

(iv) Type 3 Grammar

① Type 0 Grammar (or) unrestricted Grammar (no restriction)

→ There is no restriction for the left hand side production and right hand side production.

→ Unrestricted Grammar generates Recursive Enumerable Language.

→ This language is Accepted by turing machine.

→ The production rule, is represented by $\alpha \to \beta$

$(\alpha, \beta) \in (V \cup T)^*$

V → Non-Terminal

T → Terminal

Ex: $ab \to bcDE$

② Type - 1 Grammar (or) context sensitive grammar (CSG)

→ This Grammar generates context sensitive language. (CSL)

→ This Language is accepted by Linear Bounded Automata (LBA)

$$\alpha \rightarrow \beta$$

$$\alpha, \beta \in (V \cup T)^+$$

minimum one occurance.

(i) Length of the left production (α) is less than the length of the right side production (β)

$$|\alpha| \leq |\beta|$$

(ii) Right side production is not equal to epsilon ($\beta \neq \epsilon$)

(iii) Left hand side contains minimum one non-terminal.

eg: $A \rightarrow B$ (valid) $A \rightarrow \epsilon$ (Invalid) $aA \rightarrow B$ (Invalid)

$\beta \neq \epsilon$ $|\alpha| \leq |\beta|$

$aA \rightarrow Bacd$ (valid)

③ Type - 2 Grammar (or) context Free Grammar (CFG)

→ This Grammar generates context Free Language (CFL)

→ This Language is accepted by Push Down Automata (PDA)

→ Production is represented as $A \rightarrow \alpha$.

A is single non-terminal

where as α is combination of terminal & non-terminal

we can also use zero occurances i.e., ϵ

eg: $A \rightarrow \epsilon$ $BA \rightarrow abc$ (Invalid) $A \rightarrow abc$ $A \rightarrow abcD$

↓

Left side only one terminal

④ Type - 3 Grammar (or) Regular Grammar:

→ This Grammar generates Regular Language.

→ This language is accepted by Finite Automata (FA)

There are 2 types of Regular Grammar:

① Left-Linear Grammar: The left-most symbol in the right hand side of production rule is non-terminal.

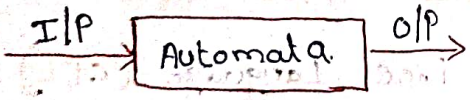
$A \longrightarrow B\ \alpha$ (B: non-terminal; α: terminal i.e., lower case)

② Right-Linear Grammar: The right-most symbol in the right hand side of production rule is non-terminal.

$A \longrightarrow \alpha\ B$ (α: terminal; B: non-terminal)

Automata:

→ Automata Theory is how different types of computational models are solved by using these automata with the help of algorithms.



Types of Automata:

→ Finite Automata (FA)
→ Push Down Automata (PDA)
→ Linear Bounded Automata (LBA)
→ Turing Machine (TM)

Applications of Finite Automata:

→ FA: Lexical analysis, Text Editors, spell checkers, Combinational & sequential circuits
→ PDA: syntax analysis, implementation of stack analysis, TOH (Towers of Hanoi) Problem

LBA: Genetic Programming, symmantic analysis of a computer.

TM: Neural networks, AI Problems, Robotic apps, Recursive Problem solving.

Terminology in Language:

① Prefix:

→ The first letter of each string is fixed. But ending may be anywhere.

Ex: ω = abcd

Prefix(ω) = {ε, a, ab, abc, abcd}

② suffix:

→ The last letter of each string is fixed. But start may be anywhere

ω = abcd

suffix(ω) = {ε, d, cd, bcd, abcd}

③ subsequence:

→ Starting and Ending is anywhere.

ω = abcd

subsequence(ω) = {ε, a, b, c, d, ab, ac, bc, ad, bd, cd, abc, abd, bcd, abcd}

④ Substring:

→ Letters of consecutive symbols

ω = abcd.

substring(ω) = {ε, a, b, c, d, ab, bc, cd, abc, bcd, abcd}

⑤ concatenation:

→ concatenation of two strings ω_1 and ω_2.

→ ω_1 is followed by ω_2 without any space b/w them.

Ex: ω_1 = Success ω_2 = fully

$\omega_1.\omega_2$ = successfully

⑥ Proper substring:

→ All substrings of ω except itself.

Ex: FLAT

Proper substrings : {ε, F, L, A, T, FL, LA, AT, FLA, LAT}

⑦ Palindrome: can be read ω either side identically.

Ex: Σ = {0, 1}

ω = 0110, ω = 1001 ω = 10001

⑧ Intersection (∩): Common elements from the given sets.

Ex: A = {1, 2, 3, 4, 5} B = {2, 6, 7, 8}

A ∩ B = {2}

⑨ union (∪): unique elements from the given sets.

Ex: A = {1, 2, 3, 4, 5} B = {2, 6, 7, 8}

A ∪ B = {1, 2, 3, 4, 5, 6, 7, 8}

⑩ cartesian Product (*): Multiply each element of one set by another element of another set.

Ex: A = {1, 2} B = {a, b}

A × B = {(1, a), (1, b), (2, a), (2, b)}

Finite Automata.

→ Finite automata are used to recognize patterns.
→ It takes the string of symbol as input and changes its state accordingly.
→ when the desired symbol is found, then the transition occurs.
→ At the time of transition, the automata can either move to the next state (or) stay in the same state.
→ Finite automata have two states, Accept state (or) Reject state.
→ when the input string is processed successfully, and the automata reached its final state, then it will accept.
→ In other words, a FA is a 5-tuple system.

$M = (Q, \Sigma, \delta, q_0, F)$ where.

Q : Finite set of states

Σ : Input alphabet

δ : Transition function mapping

$\delta : Q \times \Sigma \rightarrow Q$

q_0 : Initial state.

F : Set of Final states.

Structural representation of Finite Automata:

→ A finite automata (FA) can be represented structurally as a directed graph, also called a state transition diagram (or) state machine diagram.
→ Finite automata is represented in two ways.

① State Diagram ② Transition state Table.

State Diagram:

→ The graph consists of vertices (nodes) and edges.
→ The vertices represent the states of the machine and the edges represent the transitions between states based on input symbols.

→ The FA has one initial state, represented by an arrow pointing to the state.

→ The initial state is where the machine begins processing input symbols.

→ The machine may have one (or) more accepting states (or) Final states. indicated by a double circle around the state.

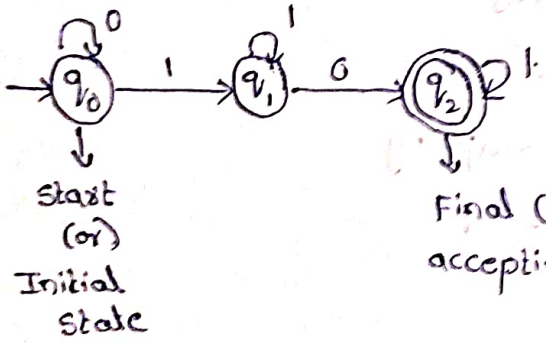
→ The Edges of the graph represent transitions between states.

→ Each edge is labeled with an input symbol, indicating the symbol that causes the machine to transition from one state to another.

→ Initial states are denoted by →○

→ Final states is denoted by ◎

Eg:



$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

$\delta = Q \times \Sigma \rightarrow Q$

(current state, next state)

$\delta = q_0 \times 0 \rightarrow q_0$

$\delta = q_0 \times 1 \rightarrow q_1$

$q_1 \times 0 \rightarrow q_2$

$q_1 \times 1 \rightarrow q_1$

q_0 = initial state = q_0

F = Final state = q_2

⑤ Transition Table:

→ It is basically a tabular representation of the transition function that takes two arguments (a state and a symbol) and returns a value (the next state).

→ Rows corresponds to states.

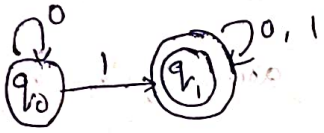
→ Columns corresponds to input symbols.

→ Entries corresponds to next states.

→ The start state is marked with an arrow (→)

→ The accept state (or) Final state is marked with a star(*) (or) Double circle (◎)

Ex:



Transition Table:

	0	1
$\rightarrow q_0$	q_0	q_1
◎ q_1	q_1	q_1

Construction of DFA:

① construct DFA that accept string start with '0'.

Step ①: write the language.

$L = \{0, 01, 00, 001, 011, 0001, \ldots\}$

Step ②: minimum length of string

Minimum length of string = 1

∴ no. of states = 1+1 = 2.

Step The states are: q_0, q_1

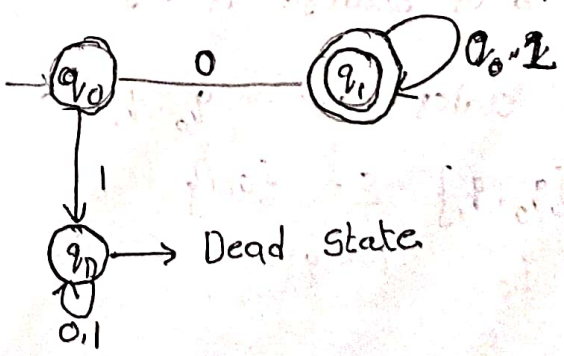
q_0 = initial state

q_1 = final state

$Q = \{q_0, q_1\}$ $\Sigma = \{0, 1\}$

$q_0 = q_0$ $F = q_1$

Step ③: Construct the DFA



Step ④ : write the transition function.

$\delta(q_0, 0) = q_0 \times 0 = q_1$

$\delta(q_0, 1) = q_0 \times 1 = q_D$

$\delta(q_1, 0) = q_1 \times 0 = q_1$

$\delta(q_1, 1) = q_1 \times 1 = q_1$

Step ⑤ : Draw the transition table.

	0	1
q_0	q_1	q_D
q_1	q_1	q_1

Step ⑥ : write the Finite automata.

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, q_1)$

② construct DFA that accept string End with '0'.

Language = {0, 10, 00, 110, 100, 010, ...}

Minimum length = 01

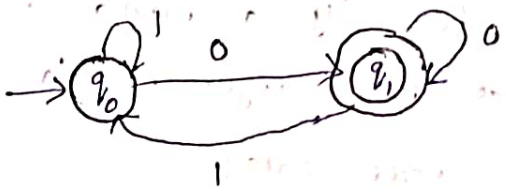
∴ The no. of states = 1+1 = 2

the states are = q_0, q_1

$Q = \{q_0, q_1\}$ $\Sigma = \{0, 1\}$

$q_0 = q_0$ $F = q_1$

construct the DFA



③ construct DFA that accept string start with 01.

Language = {01, 010, 011, 0110, 0111, ...}

Minimum length = 2

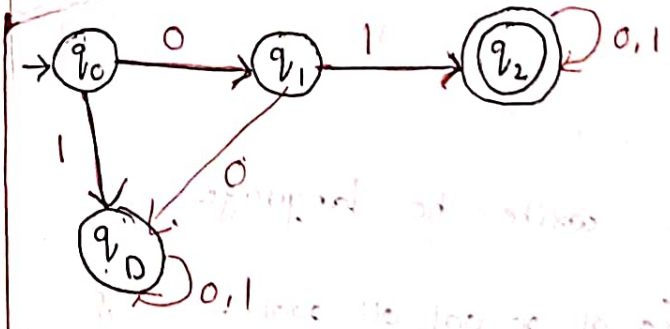
∴ The no. of states = 2+1 = 3

The states are = q_0, q_1, q_2

$Q = \{q_0, q_1, q_2\}$ $\Sigma = \{0, 1\}$

$q_0 = q_0$ $F = q_2$

construct the DFA



Transition Table:

	0	1
q_0	q_1	q_D
q_1	q_D	q_2
q_2	q_2	q_2

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, q_2)$

(4) construct DFA that accept string End with 01.

Language = {01, 001, 101, 0001, 1101, ...}

Minimum length = 2.

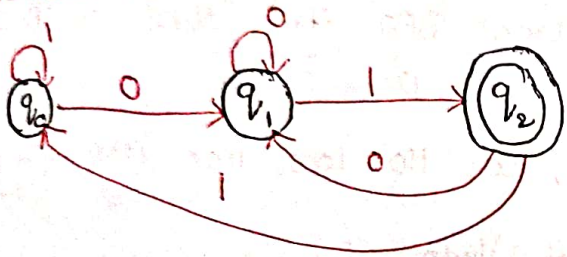
∴ The no. of states = 2+1 = 3

The states are = q_0, q_1, q_2.

$Q = \{q_0, q_1, q_2\}$ $\Sigma = \{0, 1\}$ $q = q_0$

$F = q_2$.

Construct the DFA



Transition Table

	0	1
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_0

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, q_2)$

(5) construct DFA that accept string start with 10.

Language = {10, 100, 101, 1001, 1010, 1011 ....}

Minimum length = 2

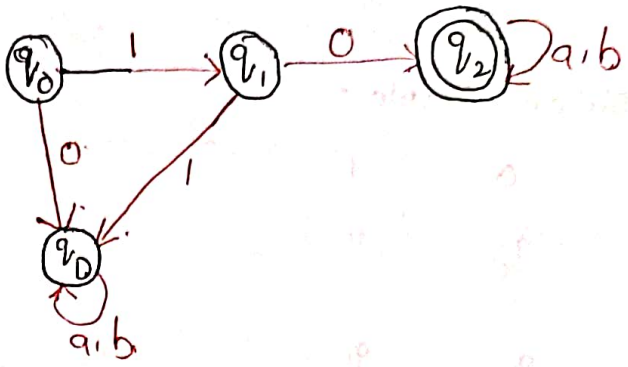
∴ The no. of states = 2+1 = 3

The states are = q_0, q_1, q_2

$Q = \{q_0, q_1, q_2\}$ $\Sigma = \{0, 1\}$ $q = q_0$

$F = q_2$

Construct the DFA



Transition Table :

	0	1
q_0	q_D	q_1
q_1	q_2	q_D
q_2	q_2	q_2

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, q_2)$

(6) construct DFA that accept string End with 10.

Language = {10, 010, 110, 0110, 0010, ...}

Minimum length = 2.

∴ The no. of states 2+1 = 3

∴ The states are = q_0, q_1, q_2.

$Q = \{q_0, q_1, q_2\}$ $\Sigma = \{0, 1\}$, $q = q_0$,

$F = q_2$.

Construct DFA

Transition Table :

	0	1
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_0	q_1

① construct DFA that ends with 00 with input symbols.

$L = \{00, 100, 000, 0100, \ldots\}$

Minimum length = 2

∴ The no of states = 2+1 = 3

∴ The states are = q_0, q_1, q_2

$Q = \{q_0, q_1, q_2\}$ $\Sigma = \{0, 1\}$ $q = q_0$ $F = q_2$

construct DFA

Transition table :

	0	1
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_0

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, q_2)$

② construct DFA that start with 1 and End with 0

$L = \{10, 100, 110, 1000, 1100, 1110, \ldots\}$

Minimum length = 2

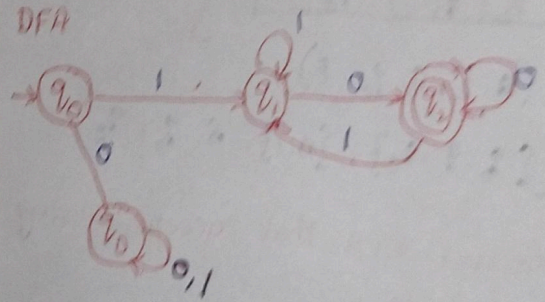
∴ The no of states = 2+1 = 3

∴ The states are = q_0, q_1, q_2

$Q = \{q_0, q_1, q_2\}$ $\Sigma = \{0, 1\}$ $q = q_0$

$F = q_2$

DFA



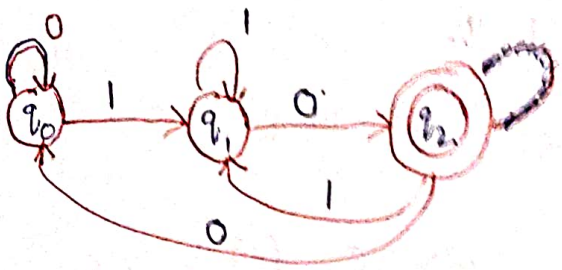
Transition Table :

	0	1
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_2	q_1

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, q_2)$

Construct DFA



Transition Table :

	0	1
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_0	q_1

(7) Construct DFA that ends with 00 with input symbols.

$L = \{00, 100, 000, 0100, \dots\}$

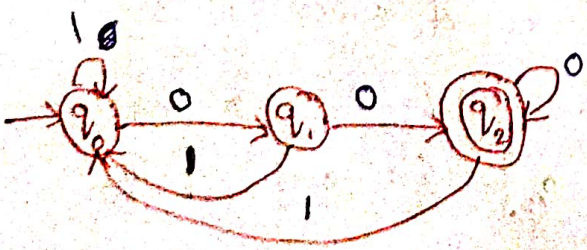
Minimum length = 2

∴ The no. of states = 2+1 = 3

∴ The states are = q_0, q_1, q_2

$Q = \{q_0, q_1, q_2\}$ $\Sigma = \{0,1\}$ $q = q_0$ $F = q_2$

Construct DFA



Transition Table :

	0	1
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_0

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1, q_2\}, \{0,1\}, \delta, q_0, q_2)$

(8) Construct DFA that start with 1 and End with 0.

$L = \{10, 100, 110, 1000, 1100, 1110, \dots\}$

Minimum length = 2

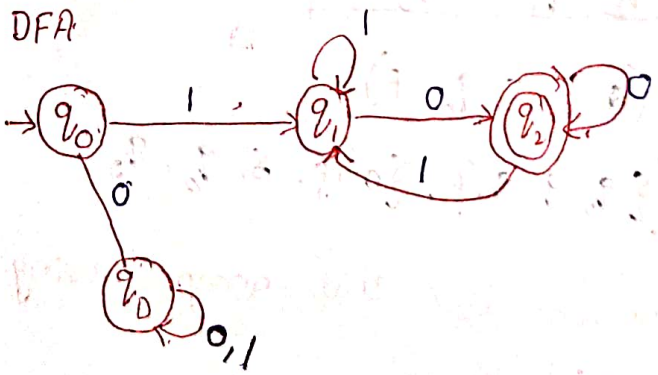
∴ The no. of states = 2+1 = 3

∴ The states are = q_0, q_1, q_2

$Q = \{q_0, q_1, q_2\}$ $\Sigma = \{0,1\}$ $q = q_0$

$F = q_2$

DFA



Transition Table :

	0	1
q_0	q_D	q_1
q_1	q_2	q_1
q_2	q_2	q_1

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1, q_2\}, \{0,1\}, \delta, q_0, q_2)$

① construct DFA that start with 0 and End with 1.

$L = \{01, 011, 001, 0011, 0111, \ldots\}$

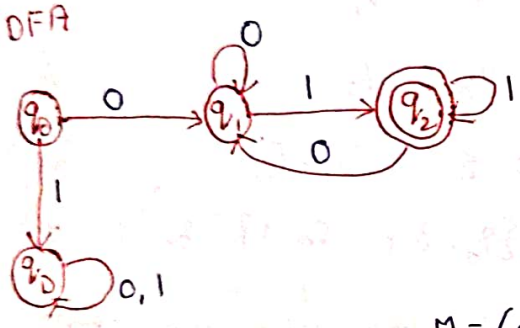
Minimum length = 2

∴ The no. of states = 2+1 = 3

∴ The states are = q_0, q_1, q_2

$Q = \{q_0, q_1, q_2\}$ $\Sigma = \{0,1\}$, $q = q_0$ $F = q_2$

DFA



Transition Table:

	0	1
→ q_0	q_1	q_D
q_1	q_1	q_2
(*) q_2	q_1	q_2

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1, q_2\}, \{0,1\}, \delta, q_0, q_2)$

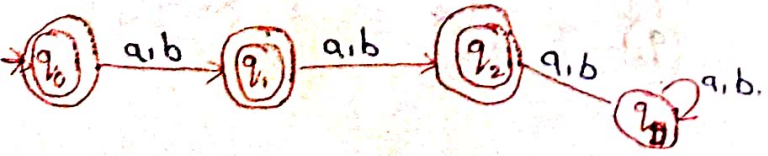
② construct DFA for the following.

a) Length = 2 b) Length <= 2

c) Length >= 2 with $\Sigma = \{a,b\}$

a) Length <= 2

$L = \{\varepsilon, a, b, aa, ab, ba, bb\}$



Transition Table

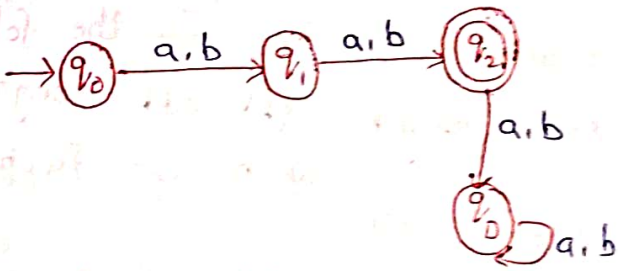
	a	b
→ q_0	q_1	q_1
q_1	q_2	q_2
(*) q_2	q_D	q_D

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1, q_2\}, \{0,1\}, \delta, q_0, q_2)$

b) Length = 2

$L = \{ab, ba, aa, ab\}$



Transition Table:

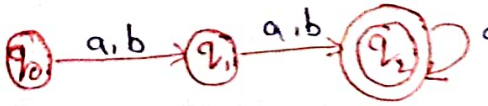
	a	b
→ q_0	q_1	q_1
q_1	q_2	q_2
(*) q_2	q_D	q_D

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1, q_2\}, \{a,b\}, \delta, q_0, q_2)$

c) Length >= 2

$L = \{aa, ab, ba, bb, abb, aab, \ldots\}$



Transition Table :

	a	b
q_0	q_1	q_1
q_1	q_2	q_2
q_2	q_2	q_2

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, q_2)$

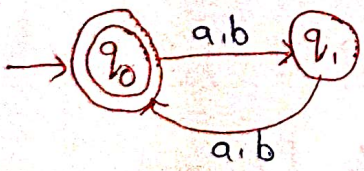
(11) construct the DFA for the following.

(a) Even length (b) odd length with $\Sigma = \{a, b\}$

(a) Even length

$L = \{\varepsilon, ab, aa, ba, bb, aabb, aaab, bbba, \ldots\}$

DFA :



Transition Table :

	a	b
q_0	q_1	q_1
q_1	q_0	q_0

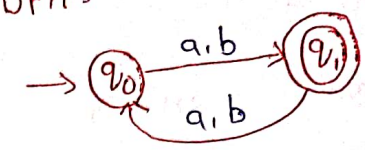
$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1\}, \{a, b\}, \delta, q_0, q_0)$

(b) odd length

$L = \{a, b, aab, aba, aaa, abb, \ldots\}$

DFA :



Transition Table :

	a	b
q_0	q_1	q_1
q_1	q_0	q_0

$M = (Q, \Sigma, \delta, q_0, F)$

$M = (\{q_0, q_1\}, \{a, b\}, \delta, q_0, q_0)$

(12) construct the DFA for that accept string containing substring

(a) Substring ab

(b) Substring aab

(c) Substring abb

(d) Substring 0

(e) Substring 00

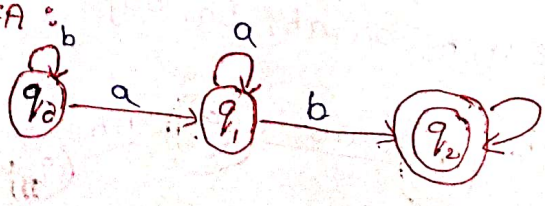
(f) Substring 101

(g) Substring 110

(a) Substring ab

$L = \{ab, abb, aab, abab, baba, \ldots\}$

DFA :

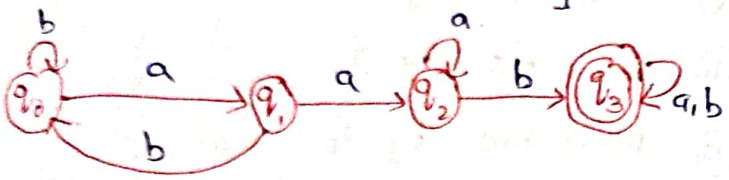


Transition Table:

	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_2	q_2

(b) substring aab

L = {aab, aabb, aaba, abaab, aaaba, ...}

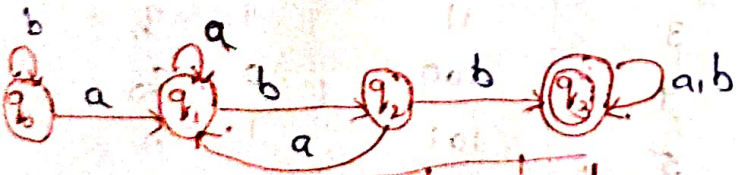


Transition Table:

	a	b
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_3
q_3	q_3	q_3

(c) substring abb

L = {abb, aabb, babb, aabba, abbb, ...}

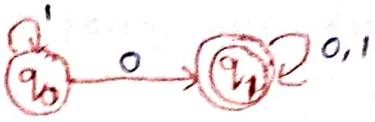


Transition Table:

	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_3
q_3	q_3	q_3

(d) substring 0

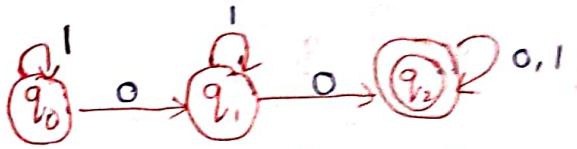
L = {0, 01, 10, 100, 000, 00, 0111...}



	0	1
q_0	q_1	q_0
q_1	q_1	q_1

(e) substring 00

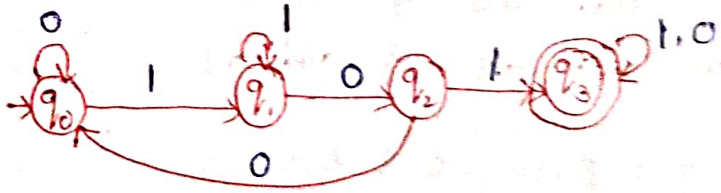
L = {00, 100, 0100, 000.....}



	0	1
q_0	q_1	q_0
q_1	q_2	q_1
q_2	q_2	q_2

(f) substring 101

L = {101, 1010, 0101, 0101, ...}

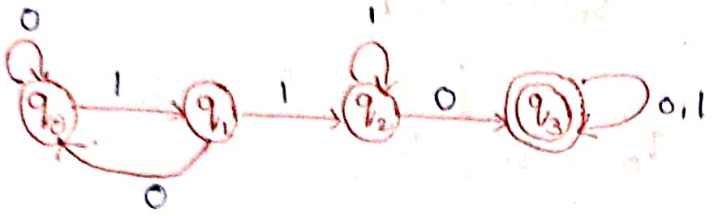


Transition Table:

	0	1
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_0	q_3
q_3	q_3	q_3

(g) substring 110

$L = \{110, 0110, 1110, 01101, \ldots\}$



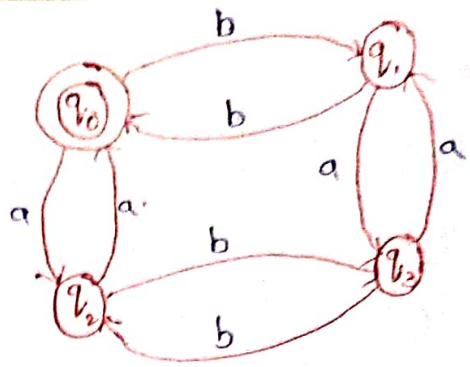
Transition Table:

	0	1
q_0	q_0	q_1
q_1	q_0	q_2
q_2	q_3	q_2
q_3	q_3	q_3

(h) construct the DFA with the,

(i) Even a's & Even b's
(ii) Even a's & odd b's
(iii) odd a's & Even b's
(iv) odd a's & odd b's

q_0 = Even a's & even b's
q_1 = Even a's & odd b's
q_2 = odd a's & Even b's
q_3 = odd a's & odd b's



(i) construct DFA with the following

(a) Divisible by 2
(b) Divisible by 3
(c) Divisible by 4
(d) Divisible by 5.

(a) Divisible by 2

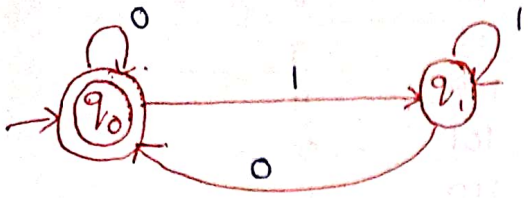
$L = \{\omega / \omega \bmod 2 = 0\}$

$L = \{00, 10, 100, 110, 1000, \ldots\}$

Decimal number	Binary number	Remainder
0	00	0 = q_0
1	01	1 = q_1
2	10	0 = q_0
3	11	1 = q_1
4	100	0 = q_0
5	101	1 = q_1
6	110	0 = q_0
7	111	1 = q_1
8	1000	0 = q_0

q_0 → Remainder 0

q_1 → Remainder 1



Transition Table :

	0	1
→ q_0	q_0	q_1
q_1	q_0	q_1

⑥ Divisible by 3

$L = \{\omega / \omega \bmod 3 = 0\}$

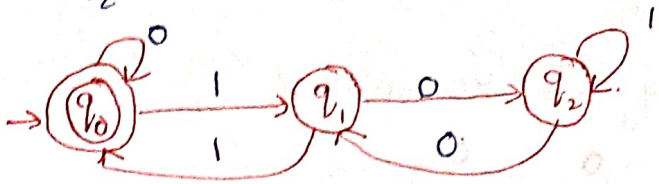
$L = \{00, 11, 110, 1001, \ldots\}$

Decimal number	Binary number	Remainder
0	00	0 = q_0
1	01	1 = q_1
2	10	2 = q_2
3	11	3 = q_0
4	100	1 = q_1
5	101	2 = q_2
6	110	0 = q_0
7	111	1 = q_1
8	1000	2 = q_2
9	1001	0 = q_0

q_0 → Remainder 0

q_1 → Remainder 1

q_2 → Remainder 2



Transition Table :

	0	1
→ q_0	q_0	q_1
q_1	q_2	q_0
q_2	q_1	q_2

⑥ Divisible by 4

$L = \{\omega / \omega \bmod 4 = 0\}$

$L = \{00, 100, 1000, 1100, \ldots\}$

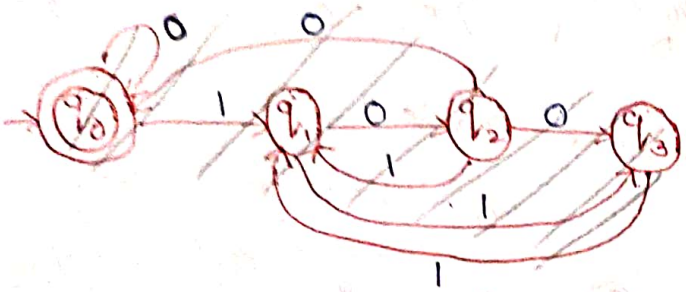
Decimal number	Binary number	Remainder
0	00	0 = q_0
1	01	1 = q_1
2	10	2 = q_2
3	11	3 = q_3
4	100	0 = q_0
5	101	1 = q_1
6	110	2 = q_2
7	111	3 = q_3
8	1000	0 = q_0
9	1001	1 = q_1
10	1010	2 = q_2

q_0 → Remainder 0

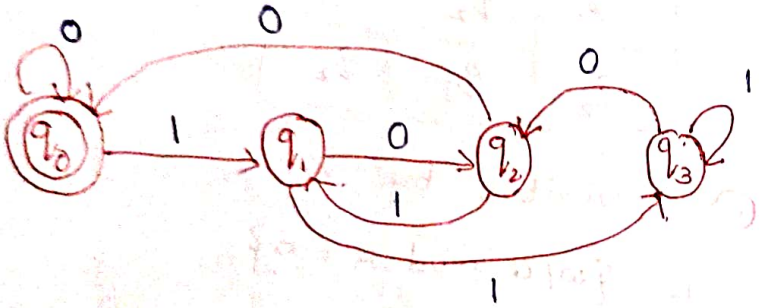
q_1 → Remainder 1

q_2 → Remainder 2

q_3 → Remainder 3



Transition Table:



Transition Table:

	0	1
q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_0	q_1
q_3	q_2	q_3

(d) Divisible by 5

$L = \{\omega \mid \omega \bmod 5 = 0\}$

$L = \{101, 1010, \text{----------}\}$

Decimal number	Binary number	Remainder
0	00	0
1	01	1
2	10	2
3	11	3
4	100	4
5	101	0
6	110	1
7	111	2
8	1000	3
9	1001	4
10	1010	0

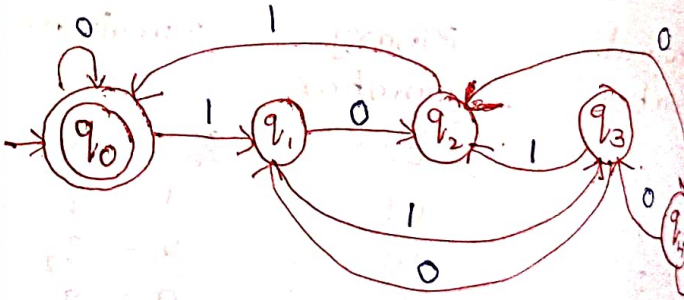
q_0 → Remainder 0

q_1 → Remainder 1

q_2 → Remainder 2

q_3 → Remainder 3

q_4 → Remainder 4



Transition Table:

	0	1
q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_4	q_0
q_3	q_1	q_2
q_4	q_3	q_4

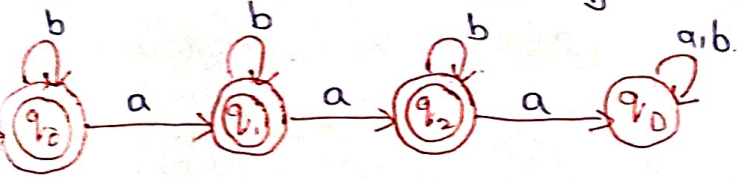
(15) construct the DFA for the following.

(a) at most 2 a's (not more than 2 a's)

(b) at most 3 a's (not more than 3 a's)

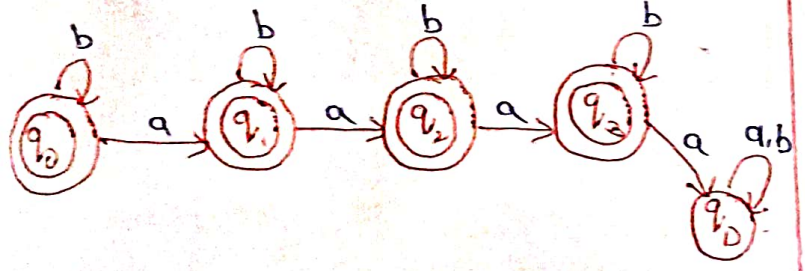
(a) at most 2 a's

$L = \{\varepsilon, ab, abb, aab, aabb, aabbb, \ldots\}$



(b) at most 3 a's

$L = \{\varepsilon, ab, a, aab, aabb, aaab, aaabb, \ldots\}$



(16) construct DFA for the following.

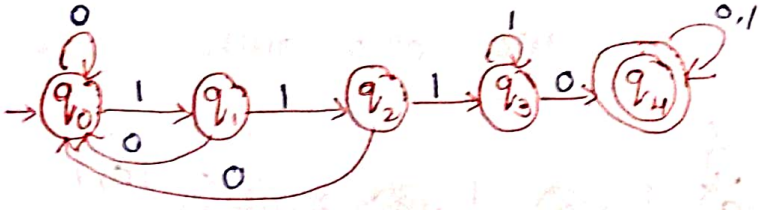
(a) except substring "aab"

(b) does not contain three consecutive 1's

(c) sub string 1110

(c) substring 1110

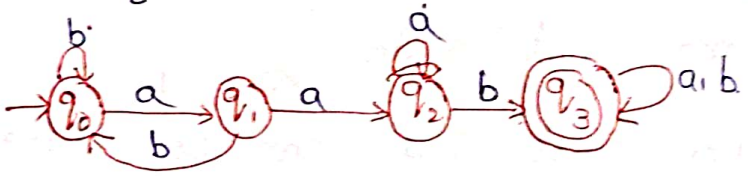
$L = \{1110, 11101, 011100, \ldots\}$



(a) except substring "aab"

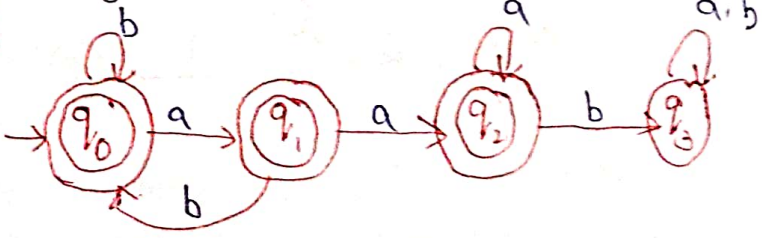
First construct the DFA for aab

$L = \{abab, abba, aa, bb, ab, aba, \ldots\}$



Except substring can be changes to Final state q_3 is changes to non-Final state.

non-Final states (q_0, q_1, q_2) are changes to Final states.

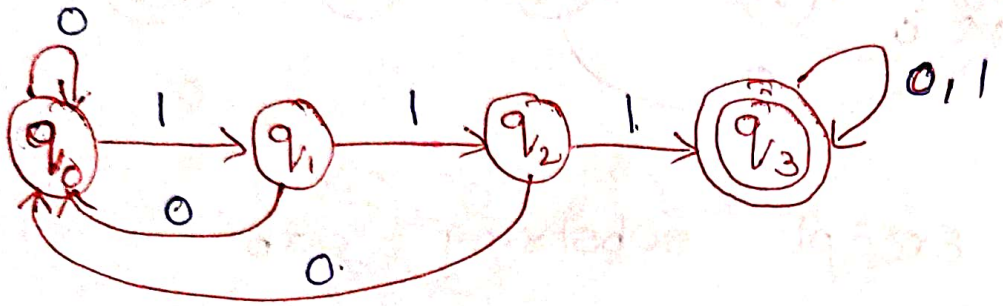


(b) does not contain three consequtive 1's

$L = \{0011, 01101, 00101, \ldots\}$

First construct the DFA for three consequtive 1's.

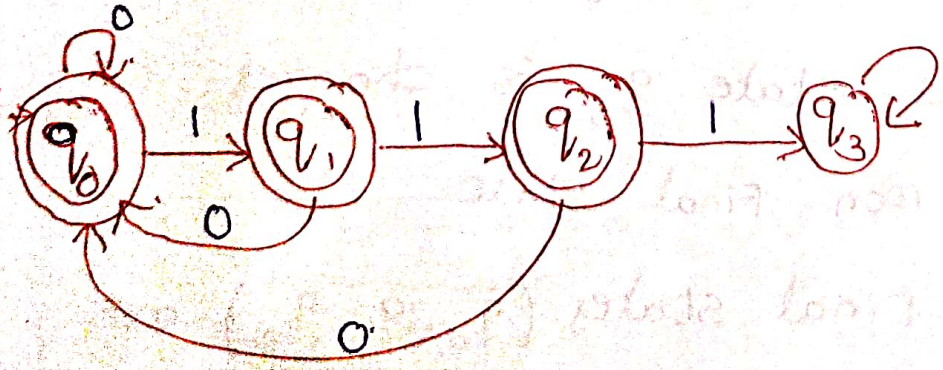
$L = \{111, 1110, 01110, 01111, \ldots\}$



does not contain three consequtive is. can be written as

Final state can be changes to non-Final state.

non-Final state can be changes to Final state.



NFA (non-Deterministic Finite Automata): 5 tuple structure

$M = (Q, \Sigma, \delta, q_0, F)$

$\delta = Q \times \Sigma \to 2^Q$

$\delta(q_0, a) = q_0$

$\delta(q_0, b) = q_1$

$\delta(q_0, b) = q_0$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$

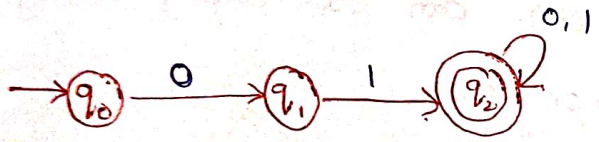
q_0 = initial state

F = Final state.

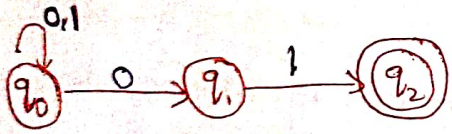
For a particular input symbol, we can have multiple symbols (edges)

construction of NFA:

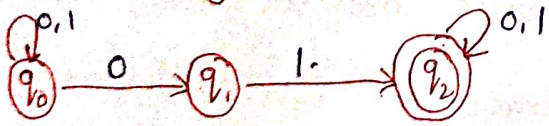
① start with 01.



② End with 01

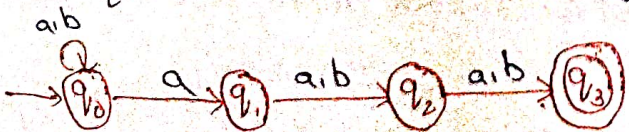


③ substring 01.



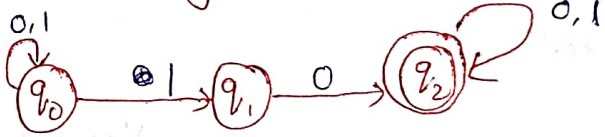
④ Third symbol from right End. is 'a'

$L = \{aaa, aab, abb, \dots\dots\}$



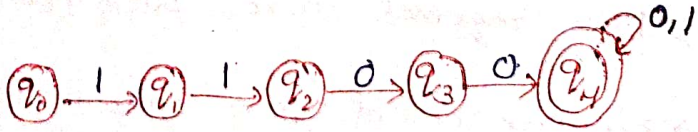
⑤ start with 10.

⑥ Substring 10



⑦ start with 1 & End with 0

⑧ start with 1100 (double 1 is followed by double 00)



Difference between DFA & NFA

DFA

→ Deterministic Finite Automata

→ There is only one fixed transition for a given input alphabet from current state to next state.

Ex: 

→ $\delta: Q \times \Sigma \rightarrow Q$

→ DFA cannot use Empty string transition.

→ In DFA, the next possible state is fixed.

→ Epsilon move is not allowed in DFA.

→ All DFA's are NFA's

→ Backtracking is allowed.

→ construction of DFA is very Difficult

→ DFA requires more space & co

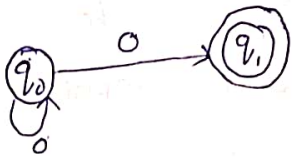
→ construction of DFA take more time.

→ Dead State is allowed in DFA.

NFA

→ Non-Deterministic Finite Automata

→ There are more than one transition for a given input symbol from current state to next state.



→ $\delta: Q \times \Sigma \rightarrow 2^Q$

→ NFA can use Empty string transition.

→ In NFA, the input symbol can have many possible next states.

→ Epsilon move is allowed in NFA

→ All NFA's are not DFA

→ Backtracking is not allowed.

→ construct of NFA is very easy

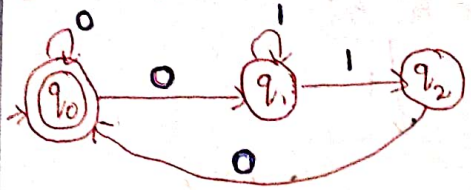
→ NFA requires less space.

→ construction of NFA take less time.

→ Dead State is not allowed in NFA.

Converting NFA to DFA

① Design DFA for the following NFA.



step①: construct the transition table, for given NFA.

	0	1
→q_0	$\{q_0, q_1\}$	$\emptyset$
q_1	$\emptyset$	$\{q_1, q_2\}$
q_2	q_0	$\emptyset$

step②: construct the transition table for DFA

	0	1
→q_0	$\{q_0, q_1\}$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_1, q_2\}$
$\{q_1, q_2\}$	q_0	$\{q_1, q_2\}$

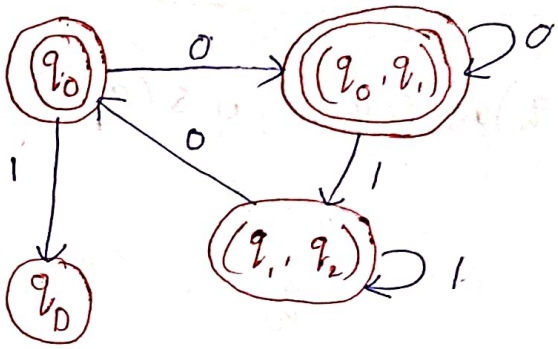
$$\delta(\{q_0, q_1\}, 0) = \delta(q_0, 0) \cup \delta(q_1, 0)$$
$$= \{q_0, q_1\} \cup \emptyset$$
$$= \{q_0, q_1\}$$

$$\delta(\{q_0, q_1\}, 1) = \delta(q_0, 1) \cup \delta(q_1, 1)$$
$$= \emptyset \cup \{q_1, q_2\}$$
$$= \{q_1, q_2\}$$

$$\delta(\{q_1, q_2\}, 0) = \delta(q_1, 0) \cup \delta(q_2, 0)$$
$$= \emptyset \cup q_0$$

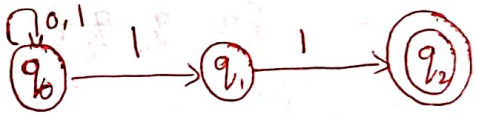
$$\delta(\{q_1, q_2\}, 0) = q_0$$

$$\delta(\{q_1, q_2\}, 1) = \delta(q_1, 1) \cup \delta(q_2, 1)$$
$$= \{q_1, q_2\} \cup \emptyset$$
$$= \{q_1, q_2\}$$



② Design DFA for the following NFA

step①: construct the transition table, for given NFA.



	0	1
→q_0	q_0	$\{q_0, q_1\}$
q_1	$\emptyset$	q_2
q_2	$\emptyset$	$\emptyset$

step ②: construct the transition table for DFA

	0	1
→ q_0	q_0	$\{q_0, q_1\}$
$\{q_0, q_1\}$	q_0	$\{q_0, q_1, q_2\}$
$\{q_0, q_1, q_2\}$	q_0	$\{q_0, q_1, q_2\}$

$\delta((q_0, q_1), 0) = \delta(q_0, 0) \cup \delta(q_1, 0)$

$= q_0 \cup \phi$

$= q_0$

$\delta((q_0, q_1), 1) = \delta(q_0, 1) \cup \delta(q_1, 1)$

$= \{q_0, q_1\} \cup q_2$

$= \{q_0, q_1, q_2\}$

$\delta((q_0, q_1, q_2), 0) = \delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0)$

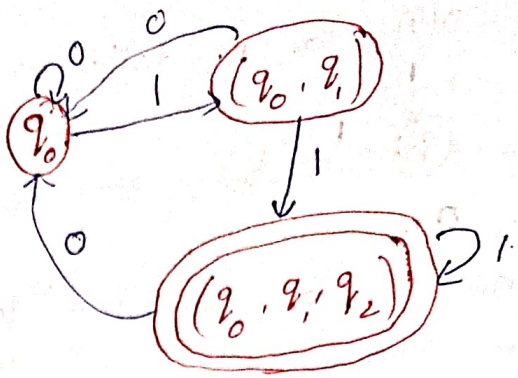
$= q_0 \cup \phi \cup \phi$

$= q_0$

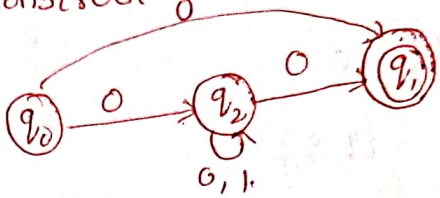
$\delta((q_0, q_1, q_2), 1) = \delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1)$

$= \{q_0, q_1\} \cup q_2 \cup \phi$

$= \{q_0, q_1, q_2\}$



③ construct the NFA to ~~NFA~~ DFA



step ①: construct transition table for NFA

	0	1
→ q_0	$\{q_0, q_2\}$	ϕ
q_1	$\{q_1, q_2\}$	q_2
q_2	ϕ	ϕ

step ②: construct transition table for DFA

	0	1
q_0	$\{q_0, q_2\}$	ϕ
$\{q_0, q_2\}$	$\{q_1, q_2\}$	q_2
q_2	$\{q_1, q_2\}$	q_2

$\delta((q_1,q_2),0) = \delta(q_1,0) \cup \delta(q_2,0)$

$= \phi \cup \{q_1,q_2\}$

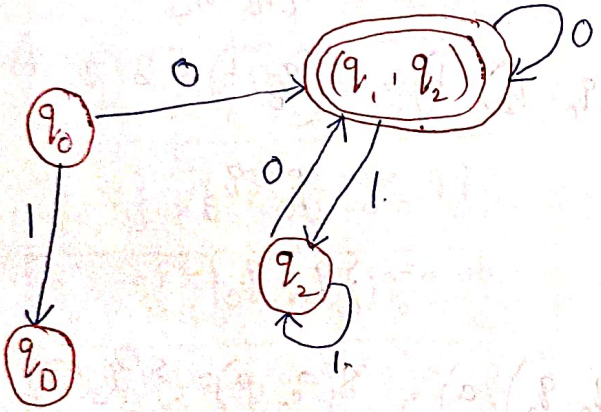
$\delta((q_1,q_2),1) = \delta(q_1,1) \cup \delta(q_2,1)$

$= \phi \cup q_2$

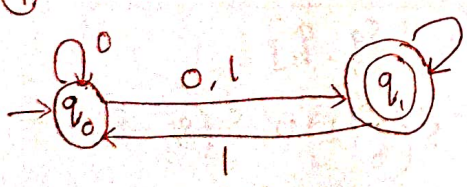
$= q_2.$

$\delta(q_2,0) = \{q_1,q_2\}$

$\delta(q_2,1) = q_2.$



(4) NFA to DFA



step ①: Transition Table

	0	1
q_0	$\{q_0,q_1\}$	q_1
q_1	ϕ	$\{q_0,q_1\}$

step ②:

	0	1
q_0	$\{q_0,q_1\}$	q_1
$\{q_0,q_1\}$	$\{q_0,q_1\}$	$\{q_0,q_1\}$
q_1	ϕ	$\{q_0,q_1\}$

$\delta((q_0,q_1),0) = \delta(q_0,0) \cup \delta(q_1,0)$

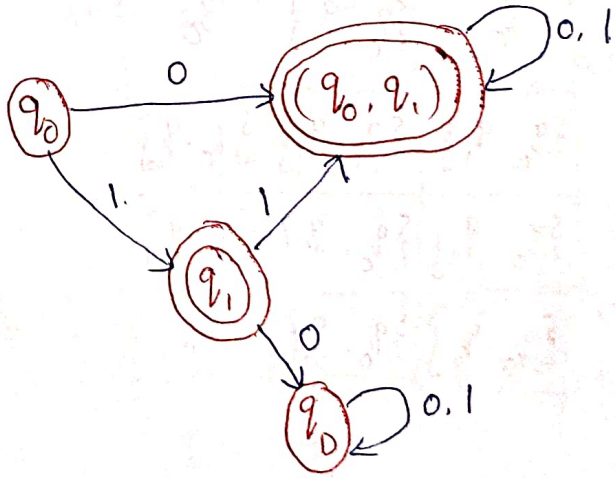
$= \{q_0,q_1\} \cup \phi$

$\delta((q_0,q_1),1) = \delta(q_0,1) \cup \delta(q_1,1)$

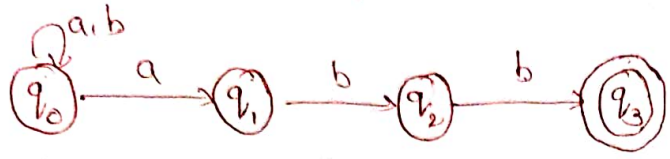
$= q_1 \cup \{q_0,q_1\}$

$\delta(q_1,\phi) = \{q_0,q_1\}$

$\delta(q_1,0) = \phi$



5 ⑤ NFA to DFA

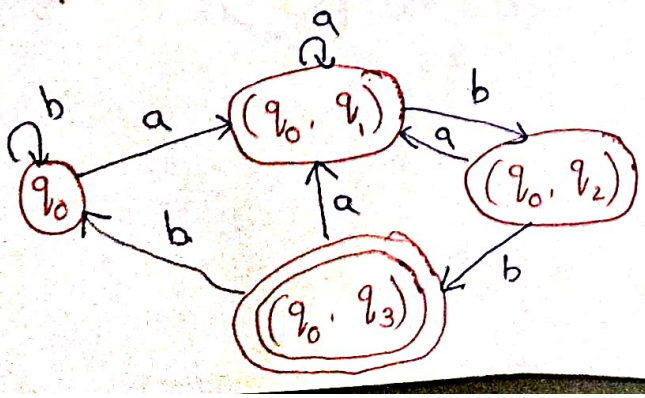


Step ①: Transition Table

	a	b
q_0	$\{q_0, q_1\}$	q_0
q_1	ϕ	q_2
q_2	ϕ	q_3
q_3	ϕ	ϕ

Step ②: Transition Table:

	a	b
q_0	$\{q_0, q_1\}$	q_0
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_3\}$
$\{q_0, q_3\}$	$\{q_0, q_1\}$	q_0



$\delta((q_0, q_1), a) = \delta(q_0, a) \cup \delta(q_1, a)$
$= \{q_0, q_1\} \cup \phi$
$= \{q_0, q_1\}$

$\delta((q_0, q_1), b) = \delta(q_0, b) \cup \delta(q_1, b)$
$= q_0 \cup q_2$
$= \{q_0, q_2\}$

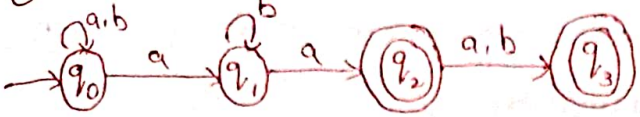
$\delta((q_0, q_2), a) = \delta(q_0, a) \cup \delta(q_2, a)$
$= \{q_0, q_1\} \cup \phi$
$= \{q_0, q_1\}$

$\delta((q_0, q_2), b) = \delta(q_0, b) \cup \delta(q_2, b)$
$= q_0 \cup q_3$
$= \{q_0, q_3\}$

$\delta((q_0, q_3), a) = \delta(q_0, a) \cup \delta(q_3, a)$
$= \{q_0, q_1\} \cup \phi$
$= \{q_0, q_1\}$

$\delta((q_0, q_3), b) = \delta(q_0, b) \cup \delta(q_3, b)$
$= q_0 \cup \phi$
$= q_0$

⑥ NFA to DFA



step ① : Transition Table.

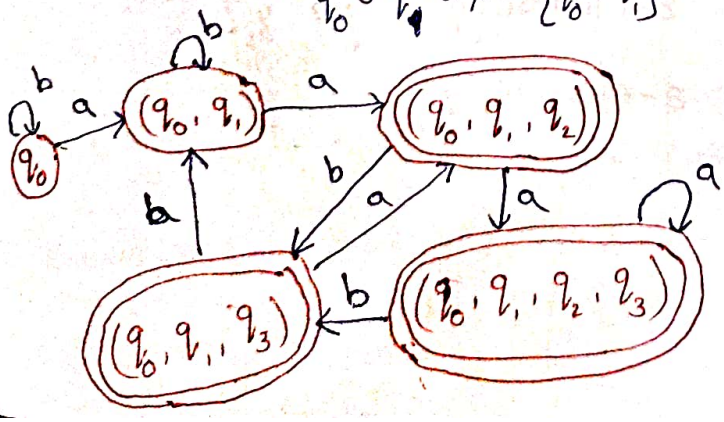
	a	b
→ q_0	$\{q_0, q_1\}$	q_0
q_1	q_2	q_1
(q_2)	q_3	q_3
(q_3)	ϕ	ϕ

step ② : Transition Table.

	a	b
q_0	$\{q_0, q_1\}$	q_0
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_1, q_3\}$
$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_1, q_3\}$
$\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$

$\delta((q_0, q_1, q_3), b) = \delta((q_0, b) \cup \delta(q_1, b) \cup \delta(q_3, b)$

$= q_0 \cup q_1 \cup \phi = \{q_0, q_1\}$



$\delta((q_0, q_1), a) = \delta(q_0, a) \cup \delta(q_1, a)$

$= \{q_0, q_1\} \cup \{q_2$

$= \{q_0, q_1, q_2\}$

$\delta((q_0, q_1), b)) = \delta(q_0, b) \cup \delta(q_1, b)$

$= q_0 \cup q_1$

$= \{q_0, q_1\}$

$\delta((q_0, q_1, q_2), a) = \delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a)$

$= \{q_0, q_1\} \cup q_2 \cup q_3$

$= \{q_0, q_1, q_2, q_3\}$

$\delta((q_0, q_1, q_2), b) = \delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b)$

$= q_0 \cup q_1 \cup q_3$

$= \{q_0, q_1, q_3\}$

$\delta((q_0, q_1, q_2, q_3), a) = \delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a) \cup \delta(q_3, a)$

$= \{q_0, q_1\} \cup q_2 \cup q_3 \cup \phi$

$= \{q_0, q_1, q_2, q_3\}$

$\delta((q_0, q_1, q_2, q_3), b) = \delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b) \cup \delta(q_3, b)$

$= q_0 \cup q_1 \cup q_3$

$= \{q_0, q_1, q_3\}$

$\delta((q_0, q_1, q_3), a) = \delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_3, a)$

$= \{q_0, q_1\} \cup q_2 \cup \phi$

$= \{q_0, q_1, q_2\}$

Epsilon NFA (ε-NFA)

→ An Epsilon non-deterministic finite automata (ε-NFA) is a type of non-deterministic finite automata (NFA) that has transitions labeled with the Greek letter epsilon (ε).

→ These transitions allow the NFA to move to another state without consuming any input symbols.

→ ε-NFAs are useful for constructing automata, especially when there are

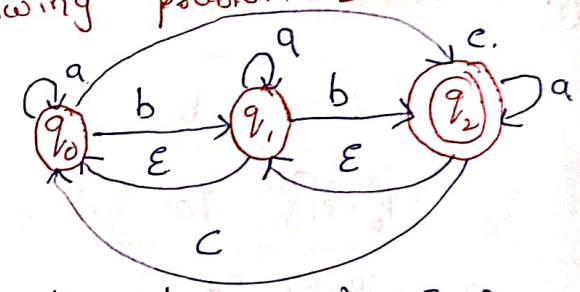
Epsilon closure (q): set of stat... that are reachable state q with... ε-transition.

converting NFA with ε-moves t... NFA without ε-moves.

→ calculate epsilon-closure of every state.

→ calculate Extended transition function (δ')

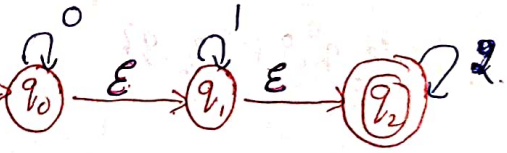
Find the epsilon closure for th... following problem · ε-NFA's.

① 

epsilon-closure (q_0) = $\{q_0\}$

epsilon-closure (q_1) = $\{q_1, q_0\}$

epsilon-closure (q_2) = $\{q_2, q_1, q_0\}$

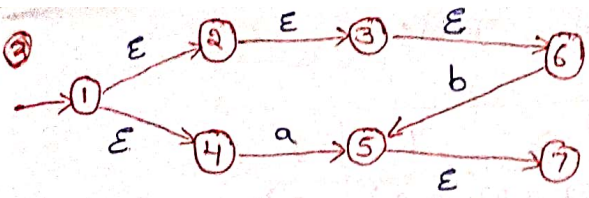
② 

ε-closure (q_0) = $\{q_0, q_1, q_2\}$

ε-closure (q_1) = $\{q_1, q_2\}$.

ε-closure (q_2) = $\{q_2\}$

③



ε-closure ε-closure (1) = {1,2,3,4,5,6}

ε-closure (2) = {2,3,6}

ε-closure (3) = {3,6}

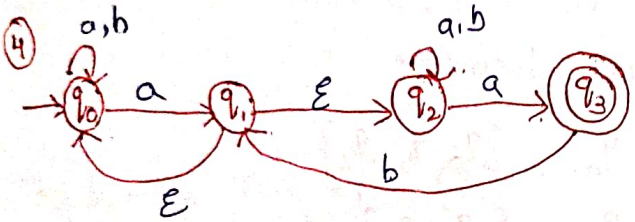
ε-closure (4) = {4}

ε-closure (5) = {5,7}

ε-closure (6) = {6}

ε-closure (7) = {7}

④



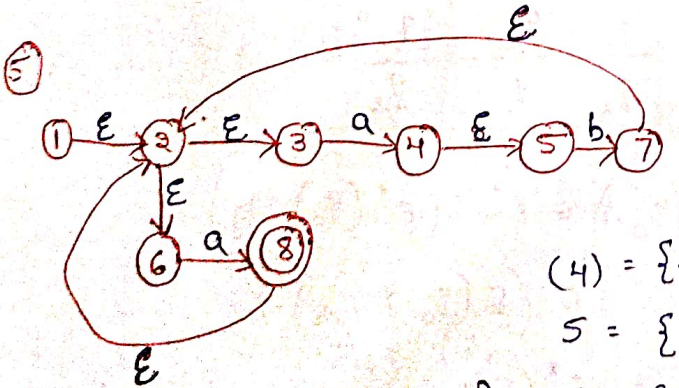
ε-closure(q_0) = {q_0}

ε-closure(q_1) = {q_1, q_2, q_0}

ε-closure(q_2) = {q_2}

ε-closure(q_3) = {q_3}

⑤



ε-closure (1) = {1,2,3,6}

(2) = {2,3,6}

(3) = {3}

(4) = {4,5}

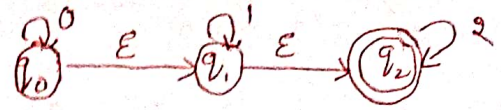
5 = {5}

6 = {6}

7 = {7,2,3,6}

8 = {8,2,3,6}

⑥ construct the NFA using Extended transition function.



step ①:

ε-closure (q_0) = {q_0, q_1, q_2}

ε-closure (q_1) = {q_1, q_2}

ε-closure (q_2) = {q_2}

step ②:

$\delta'(q_0, 0) = \varepsilon^*(\delta(\varepsilon^*(q_0), 0))$

$= \varepsilon^*(\delta(q_0, q_1, q_2), 0))$

$= \varepsilon^*(\delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0))$

$= \varepsilon^*(q_0 \cup \emptyset \cup \emptyset) = \varepsilon^*(q_0)$

$\delta'(q_0, 0) = \{q_0, q_1, q_2\}$

$\delta'(q_0, \emptyset) = \varepsilon^*(\delta(\varepsilon^*(q_0), \emptyset))$

$= \varepsilon^*(\delta(q_0, q_1, q_2), \emptyset)$

$= \varepsilon^*(\delta(q_0, \emptyset) \cup \delta(q_1, \emptyset) \cup \delta(q_2, \emptyset))$

$= \varepsilon^*(\emptyset \cup q_1 \cup \emptyset) = \varepsilon^*(q_1)$

$= \{q_1, q_2\}$

$\delta'(q_0, 2) = \varepsilon^*(\delta(\varepsilon^*(q_0), 2))$

$= \varepsilon^*(\delta(q_0, q_1, q_2), 2)$

$= \varepsilon^*(\delta(q_0, 2) \cup \delta(q_1, 2) \cup \delta(q_2, 2))$

$= \varepsilon^*(\emptyset \cup \emptyset \cup q_2) = \varepsilon^*(q_2)$

$= \{q_2\}$

$\delta'(q_1, 0) = \varepsilon^*(\delta(\varepsilon^*(q_1), 0))$

$= \varepsilon^*(\delta(q_1, q_2), 0)$

$= \varepsilon^*(\delta(q_1, 0) \cup \delta(q_2, 0))$

$= \varepsilon^*(\phi \cup \phi)$

$= \phi$

$\delta'(q_1, 1) = \varepsilon^*(\delta(\varepsilon^*(q_1), 1))$

$= \varepsilon^*(\delta(q_1, q_2), 1)$

$= \varepsilon^*(\delta(q_1, 1) \cup \delta(q_2, 1))$

$= \varepsilon^*((q_1) \cup \phi)$

$= \varepsilon^*(q_1) = \{q_1, q_2\}$

$\delta'(q_1, 2) = \varepsilon^*(\delta(\varepsilon^*(q_1), 2))$

$= \varepsilon^*(\delta(q_1, q_2), 2)$

$= \varepsilon^*(\delta(q_1, 2) \cup \delta(q_2, 2))$

$= \varepsilon^*(\phi \cup q_2) = \varepsilon^*(q_2)$

$= \varepsilon^* \{q_2\}$

$\delta'(q_2, 0) = \varepsilon^*(\delta(\varepsilon^*(q_2), 0))$

$= \varepsilon^*(\delta(q_2, 0)) = \varepsilon^*(\phi) = \phi$

$\delta'(q_2, 1) = \varepsilon^*(\delta(\varepsilon^*(q_2), 1))$

$= \varepsilon^*(\delta(q_2, 1)) = \varepsilon^*(\phi) = \phi$

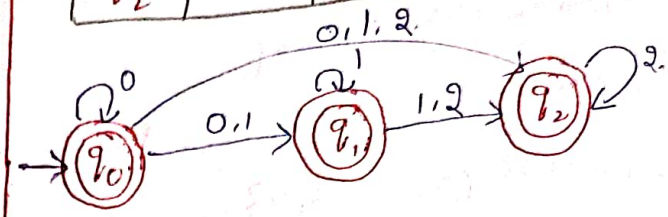
$\delta'(q_2, 2) = \varepsilon^*(\delta(\varepsilon^*(q_2), 2))$

$= \varepsilon^*(\delta(q_2, 2)) = \varepsilon^*(q_2) =$

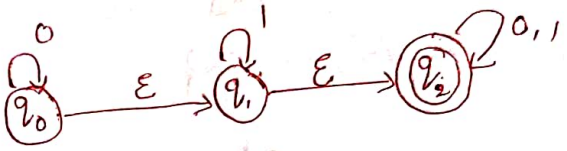
$= \{q_2\}$

Transition

	0	1	2
$\rightarrow q_0^*$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1^*	ϕ	$\{q_1, q_2\}$	q_2
q_2^*	ϕ	ϕ	q_2



(2)



Epsilon - closure :

$q_0 = \{q_0, q_1, q_2\}$

$q_1 = \{q_1, q_2\}$

$q_2 = \{q_2\}$

Transition Table for NFA

δ'	0	1
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$
q_1	q_2	$\{q_1, q_2\}$
q_2	q_2	q_2

$\delta'(q_0, 1) = \varepsilon^*(\delta(\varepsilon^*(q_0), 1))$

$= \varepsilon^*(\delta(q_0, q_1, q_2), 1)$

$= \varepsilon^*(\delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1))$

$= \varepsilon^*(\phi \cup q_1 \cup q_2)$

$= \varepsilon^*(q_1 \cup q_2) = \{q_1, q_2\}$

$\delta'(q_0, 0) = \varepsilon^*(\delta(\varepsilon^*(q_0), 0))$

$= \varepsilon^*(\delta(q_0, q_1, q_2), 0)$

$= \varepsilon^*(\delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0))$

$= \varepsilon^*(q_0 \cup q_2)$

$= \varepsilon^*(q_0) \cup \varepsilon^*(q_2)$

$= \{q_0, q_1, q_2\}$

$\delta'(q_1, 0) = \varepsilon^*(\delta(\varepsilon^*(q_1), 0))$

$= \varepsilon^*(\delta(q_1, q_2), 0)$

$= \varepsilon^*(\delta(q_1, 0) \cup \delta(q_2, 0))$

$= \varepsilon^*(\emptyset \cup q_2) = \varepsilon^*(q_2)$

$= \{q_2\}$

$\delta'(q_1, 1) = \varepsilon^*(\delta(\varepsilon^*(q_1), 1))$

$= \varepsilon^*(\delta(q_1, q_2), 1)$

$= \varepsilon^*(\delta(q_1, 1) \cup \delta(q_2, 1))$

$= \varepsilon^*(q_1 \cup q_2)$

$= \varepsilon^*(q_1) \cup \varepsilon^*(q_2) = \{q_1, q_2\}$

$\delta'(q_2, 0) = \varepsilon^*(\delta(\varepsilon^*(q_2), 0)$

$= \varepsilon^*(\delta(q_2), 0)$

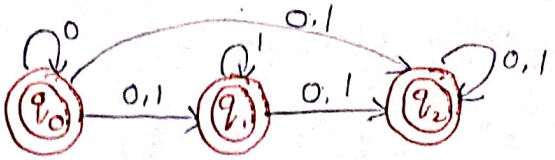
$= \varepsilon^*(q_2, 0) = \varepsilon^* \varepsilon^*(q_2)$

$= q_2$

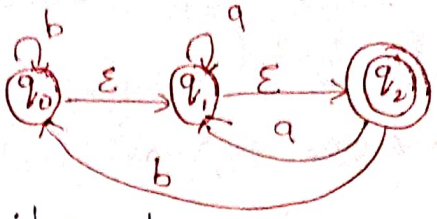
$\delta'(q_2, 1) = \varepsilon^*(\delta(\varepsilon^*(q_2), 1))$

$= \varepsilon^*(\delta(q_2, 1))$

$= \varepsilon^*(q_2, 1) = \varepsilon^*(q_2) = q_2$



③



epsilon - closure :

$q_0 = \{q_0, q_1, q_2\}$

$q_1 = \{q_1, q_2\}$

$q_2 = \{q_2\}$

$\delta(q_0, a) = \varepsilon^*(\delta(\varepsilon^*(q_0), a))$

$= \varepsilon^*(\delta(\varepsilon^* q_0, q_1, q_2), a)$

$= \varepsilon^*(\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a))$

$= \varepsilon^*(\phi \cup q_1 \cup q_1)$

$= \varepsilon^*(q_1) = \{q_1, q_2\}$

$\delta(q_0, b) = \varepsilon^*(\delta(\varepsilon^*(q_0), b))$

$= \varepsilon^*(\delta(\varepsilon^*(q_0, q_1, q_2), b))$

$= \varepsilon^*(\delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b))$

$= \varepsilon^*(q_0 \cup q_0) = \varepsilon^*(q_0) =$

$\{q_0, q_1, q_2\}$

$\delta(q_1, a) = \varepsilon^*(\delta(\varepsilon^*(q_1), a))$

$= \varepsilon^*(\delta(q_1, q_2), a))$

$= \varepsilon^*(\delta(q_1, a) \cup \delta(q_2, a))$

$= \varepsilon^*(q_1 \cup q_2) = \{q_1, q_2\}$

$\delta(q_1, b) = \varepsilon^*(\delta(\varepsilon^*(q_1), b))$

$= \varepsilon^*(\delta(q_1, q_2), b)) =$

$= \varepsilon^*(\delta(q_1, b) \cup \delta(q_2, b))$

$= \varepsilon^*(\phi \cup q_0) = \varepsilon^*(q_0)$

$= \{q_0, q_1, q_2\}$

$\delta(q_2, a) = \varepsilon^*(\delta(\varepsilon^*(q_2), a))$

$= \varepsilon^*(\delta(q_2, a)) = \varepsilon^*(q_1)$

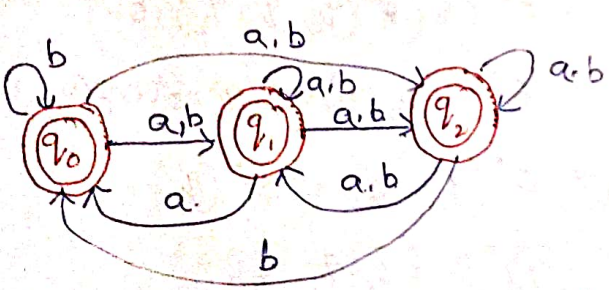
$= \{q_1, q_2\}$

$\delta(q_2, b) = \varepsilon^*(\delta(\varepsilon^*(q_2), b))$

$= \varepsilon^*(\delta(q_2, b)) = \varepsilon^*(q_0)$

$= \{q_0, q_1, q_2\}$

δ'	a	b
q_0	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$
q_1	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$
q_2	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$



(4)

Epsilon - closure :

$q_0 = \{q_1\}$

$q_1 = \{q_1, q_2\}$ $q_2 = \{q_2\}$

$\delta'(q_0, a) = \varepsilon^*(\delta(\varepsilon^*(q_0), a))$

$= \varepsilon^*(\delta(q_0), a)$

$= \varepsilon^*(q_1)$

$= \{q_1, q_2\}$

$\delta'(q_0, b) = \varepsilon^*(\delta(\varepsilon^*(q_0), b))$

$= \varepsilon^*(\delta(q_0, b))$

$= \varepsilon^*(\phi) =$

$= \phi$

$\delta'(q_1, a) = \varepsilon^*(\delta(\varepsilon^*(q_1), a))$

$= \varepsilon^*(\delta(q_1, q_2), a)$

$= \varepsilon^*(\delta(q_1, a) \cup \delta(q_2, a))$

$= \varepsilon^*(\phi) = \phi$

$\delta'(q_1, b) = \varepsilon^*(\delta(\varepsilon^*(q_1), b))$

$= \varepsilon^*(\delta(q_1, q_2), b)$

$= \varepsilon^*(\delta(q_1, b) \cup \delta(q_2, b))$

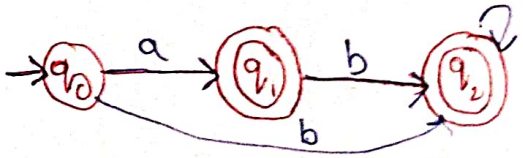
$= \varepsilon^*(\phi \cup q_2) = \varepsilon^*(q_2) = q$

$\delta'(q_2, a) = \varepsilon^*(\delta(\varepsilon^*(q_2), a))$

$= \varepsilon^*(\delta(q_2, a)) = \varepsilon^*(\phi) = \phi$

$\delta'(q_2, b) = \varepsilon^*(\delta(\varepsilon^*(q_2), b))$

$= \varepsilon^*(\delta(q_2, b)) = \varepsilon^*(q_2) = q$



Regular Expression

Regular Language: The language which is accepted by finite automata.

→ Regular expression is used to represent the Regular language.

→ Regular expression is a pattern (or) algebric expression

→ Let Σ be one input alphabet then regular expression (RE) can be defined as.

1. ϕ is R.E. which denotes empty set $\{\}$
2. ε is R.E. which denotes null string $\{\varepsilon\}$
3. If R_1 is R.E. then R^* is also R.E. $R^* \rightarrow$ Kleen closure.
4. If R_1 is R.E. then R^+ is also R.E $R^+ \rightarrow$ positive closure
5. If R_1 & R_2 are R.E. then $R_1 + R_2$ is also a R.E.
   ↓ union.
6. If R_1 & R_2 are R.E. then $R_1 \cdot R_2$ is also a R.E.
   ↓ concatenation.

Identity Rules (or) Algebric laws of Regular expressions:

1. $\phi + R = R + \phi = R \rightarrow$ null set Identity
2. $\phi R = R\phi = \phi \rightarrow$ null set multiplication Identity.
3. $\varepsilon R = R\varepsilon = R \rightarrow$ Empty string Identity.
4. $\varepsilon^* = \varepsilon$, $\phi^* = \varepsilon \rightarrow$ star of Empty string and null set.
5. $R + R = R \rightarrow$ Idempotent Law
6. $R^* R^* = R^* \rightarrow$ star multiplication Law
7. $R^* R = R R^* \rightarrow$ concatenation Law.
8. $(R^*)^* = R^* \rightarrow$ star of star Law
9. $\varepsilon + RR^* = \varepsilon + R^*R = R^* \rightarrow$ Empty string and star
10. $(P+Q)R = PR + QR \rightarrow$ Distributive law
11. $(PQ)^* P = P(QP)^*$
12. $(P+Q)^* = (P^* Q^*)^* = (P^* + Q^*)^*$

Eg:. Design a Regular expression for the following:

① All possible combinations of a's & b's are $\Sigma = \{a, b\}$

$L = \{\varepsilon, a, b, aa, ab, ba, bb, aab, aaa, \ldots\}$

$RE = (a+b)^*$

② Ends when 00 over $\Sigma = \{0, 1\}$

$L = \{00, 100, 000, 1000, 0100, 1000 \ldots\}$

$RE = (0+1)^* 00$

③ starts with 1 & end with 0

$L = \{10, 110, 100, 1010, 1100, 1000, \ldots\}$

$RE = 1(0+1)^* 0$

④ Any no. of a's followed by any no. of b's followed by any no. of c's.

$L = \{\varepsilon, abc, aabc, abbc, abcc, \ldots\}$

$RE = a^* b^* c^*$

⑤ Atleast 1a followed by atleast 1b followed by atleast 1c.

$L = \{abc, aabc, abbc, \ldots\}$

$RE = a^+ b^+ c^+$

⑥ Begins (or) ends with either 00 (or) 11

$L = \{00, 11, 011, 000, \ldots\}$

$RE = (00+11)(0+1)^* + (0+1)^*(00+11)$

⑦ Third character is a over $\Sigma = \{a, b\}$

$RE = (a+b)^* a (a+b)(a+b)$

⑧ Atleast 2 b's over $\Sigma = \{a, b\}$

$RE = (a+b)^* b (a+b)^* b (a+b)^*$

⑨ Exactly 2 b's over $\Sigma = \{a, b\}$

$RE = a^* b a^* b a^*$

⑩ Even length string over $\Sigma = \{0\}$

$RE = (00)^*$

⑪ odd length string are $\Sigma = \{1\}$

$RE = (11)^* 1$ (or) $1(11)^*$

⑫ no. of a's are divisible by 3

$RE = (b^* a b^* a b^* a b^*)^*$

⑬ at most 2 a's over $\{a, b\}$

at most 2 a's means

either a = 0, 1, 2,

$0a = b^*$ $1a = b^* a b^*$

$2a = b^* a b^* a b^*$

$RE = b^* + b^* a b^* + b^* a b^* a b^*$

⑭ 3 consecutive a's

$RE = (a+b)^* aaa (a+b)^*$

⑮ Does not contain substring ab

$RE = b^* a^*$

(16) $\{\varepsilon, 0, 00, 000, \ldots\ldots\}$

R.E. = 0^*

(17) $\{1, 11, 111, \ldots\ldots\}$

R.E. = 1^+

(18) Length = 2 over $\Sigma = \{a, b\}$

$L = \{aa, ab, ba, bb\}$

R.E. = $aa + ab + ba + bb$

$= a(a+b) + b(a+b)$

R.E. = $(a+b)(a+b)$

(19) Length = 3 over $\Sigma = \{a, b\}$

R.E. = $(a+b)(a+b)(a+b)$

(20) Even length over $\Sigma = \{a, b\}$

R.E. = $((a+b)(a+b))^*$

(21) odd length over $\Sigma = \{a, b\}$

R.E. = $((a+b)(a+b))^*(a+b)$

(22) Even no. of a's

R.E. = $(b^* a b^* a b^*)^* + b^*$

(23) odd no. of a's

R.E. = $b^* a b^* (a b^* a b^*)^*$

Finite Automata to Regular expression using Arden's theorem:

conditions:

→ Initial state compulsary.

→ No ε-moves.

steps:

① state Equation for each state based on the incoming edges

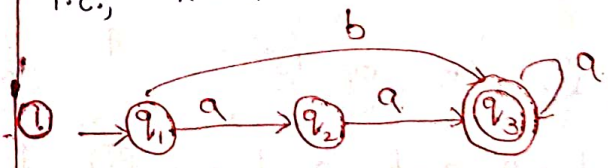
② Add ε to the initial state. state equation.

③ simplify final state state equation, and the union of 2 final state will become Regular Expression.

Arden's Theorem: If P and Q are two regular expressions over Σ, and if P does not contain ε, then the following equation in R given by

$R = Q + RP$ has a unique solution.

i.e., $R = QP^*$



State equations:

$q_1 = \varepsilon$ —①

$q_2 = q_1 \cdot a$ —②

$q_3 = q_3 \cdot a + q_2 \cdot a + q_1 \cdot b$ —③

Ex: Simplify the final set state, state equation.

sub eq ② in eq ③

$q_3 = q_3 \cdot a + q_1 \cdot b + q_1 \cdot aa$

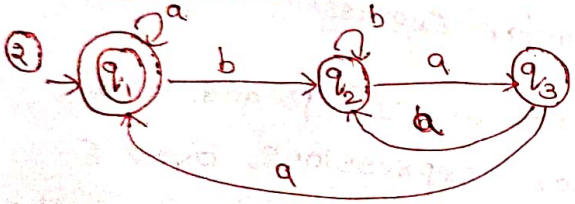
$q_3 = q_3 \cdot a + q_1 (b + aa)$

$q_3 = q_3 \cdot a + \varepsilon (b + aa)$

$R = R \cdot P + Q$. then $R = QP^*$

$q_3 = \varepsilon (b + aa) \cdot a^*$

$\boxed{q_3 = (b + aa) \cdot a^*}$



$q_1 = q_1 \cdot a + q_3 \cdot a + \varepsilon$ — ①

$q_2 = q_2 \cdot b + q_3 \cdot a + q_1 \cdot b$ — ②

$q_3 = q_2 \cdot a$ — ③

sub. eq ③ in eq ②

$q_2 = q_2 \cdot b + q_2 \cdot a \cdot a + q_1 \cdot b$ — ④

$q_2 = q_2 (b + aa) + q_1 \cdot b$

$R = RP + Q$. then $R = QP^*$

$q_2 = q_1 \cdot b (b + aa)^*$ — ⑤

sub eq ⑤ in eq. ③

$q_3 = q_1 \cdot b (b + aa)^* \cdot a$ — ⑥

sub eq ⑥ in eq. ①

$q_1 = q_1 \cdot a + q_1 \cdot b (b + aa)^* \cdot a \cdot a + \varepsilon$

$q_1 = q_1 (a + b (b + aa)^* \cdot aa) + \varepsilon$

$R = RP + Q$. then $R = QP^*$

$q_1 = \varepsilon \cdot (a + b (b + aa)^* \cdot aa)^*$

$\boxed{q_1 = (a + b (b + aa)^* \cdot aa)^*}$

③

$q_1 = q_1 \cdot 0 + \varepsilon$ — ①

$q_2 = q_1 \cdot 1 + q_2 \cdot 1$ — ②

$q_3 = q_2 \cdot 0 + q_3 \cdot 0 + q_3 \cdot 1$ — ③

eq ①

$q_1 = q_1 \cdot 0 + \varepsilon$

$R = R \cdot P + Q$ then $R = QP^*$

$q_1 = \varepsilon \cdot 0^* = 0^*$

$q_1 = 0^*$ — ④

sub. eq ④ in eq ②

$q_2 = 0^* \cdot 1 + q_2 \cdot 1$

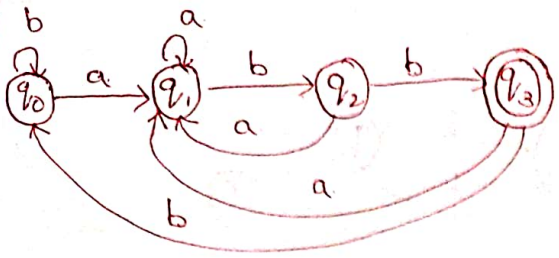
$R = QP + RP$ then $R = QP^*$

$q_2 = 0^* \cdot 1 \cdot 1^*$ — ⑤

The Final solution (Regular expression) is $R.E. = q_1 + q_2$

$\boxed{R.E. = 0^* + 0^* \cdot 1 \cdot 1^*}$

(4)



$q_0 = q_0 \cdot b + q_3 \cdot b + \varepsilon$ —— (1)

$q_1 = q_0 \cdot a + q_1 \cdot a + q_2 \cdot a + q_3 \cdot a$ —— (2)

$q_2 = q_1 \cdot b$ —— (3)

$q_3 = q_2 \cdot b$ —— (4)

sub. eq. (3) & eq (4) values in eq (2)

$q_1 = q_0 \cdot a + q_1 \cdot a + q_1 \cdot bb + q_2 \cdot b \cdot a$ —— (5)

sub eq (3) in eq. (5)

$q_1 = q_0 \cdot a + q_1 \cdot a + q_1 \cdot ba + q_1 \cdot b \cdot b \cdot a$

$q_1 = q_0 \cdot a + q_1 (a + ba + bba)$

$R = Q + R \cdot P$ then $R = QP^*$

$q_1 = q_0 \cdot a (a + ba + bba)^*$ —— (6)

sub eq (6) in eq. (3)

$q_2 = q_0 \cdot a (a + ba + bba)^* \cdot b$ —— (7)

sub (4) eq. (7) in eq (4)

$q_3 = q_0 \cdot a (a + ba + bba)^* \cdot b \cdot b$ —— (8)

sub eq. (8) in eq (1)

$q_0 = q_0 \cdot b + q_3 \cdot b$

$q_0 = q_0 \cdot b + q_0 \cdot a (a + ba + bba)^* bbb + \varepsilon$

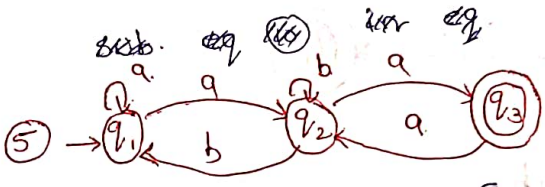
$q_0 = q_0 (b + a(a + ba + bba)^* bbb) + \varepsilon$

$R = R \cdot P + Q$ then $R = QP^*$

$q_0 = \varepsilon \cdot (b + a(a + ba + bba)^* bbb)^*$ —— (9)

sub eq. (9) in eq. (8)

$q_3 = \varepsilon \cdot (b + a(a + ba + bba)^* bbb)^* \cdot a$
$(a + ba + bba)^* bb$ —— (10)



$q_1 = \varepsilon + q_1 \cdot a + q_2 \cdot b$ —— (1)

$q_2 = q_1 \cdot a + q_2 \cdot b + q_3 \cdot a$ —— (2)

$q_3 = q_2 \cdot a$ —— (3)

sub. eq. (3) in eq (2)

$q_2 = q_1 \cdot a + q_2 \cdot b + q_2 \cdot a \cdot a$ —— (4)

$q_2 = q_1 \cdot a + q_2 (b + aa)$

$R = Q + RP$ then $R = QP^*$

$q_2 = q_1 \cdot a (b + aa)^*$ —— (5)

sub. eq (5) in eq. (1)

$q_1 = \varepsilon + q_1 \cdot a + q_1 \cdot a (b + aa)^* b$

$q_1 = \varepsilon + q_1 (a + a(b + aa)^* \cdot b)$

$q_1 = \varepsilon \cdot (a + a(b + aa)^* \cdot b)^*$ —— (6)

sub. eq (6) in eq (5)

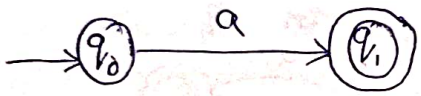
$q_2 = \varepsilon\,(a+a\,(b+aa)^*\,b)^*.\,a\,(b+aa)^*$ —⑦

sub eq ⑦ in eq ③

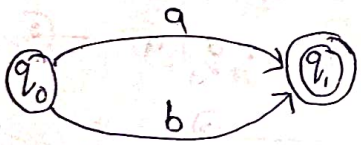
$q_3 = \varepsilon(a+a(b+aa)^*\,b^*).\,a\,(b+aa)^*.\,a$

Regular expression to Finite automata (NFA)

① $R = a$

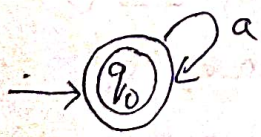


② $R = a+b$

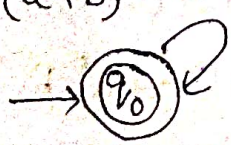


③ $R = ab$

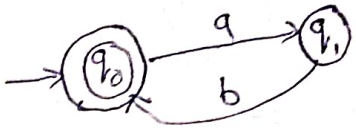
④ $R = a^*$



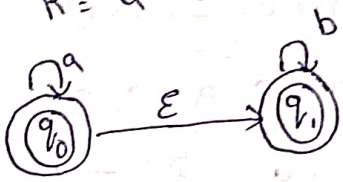
⑤ $R = (a+b)^*$



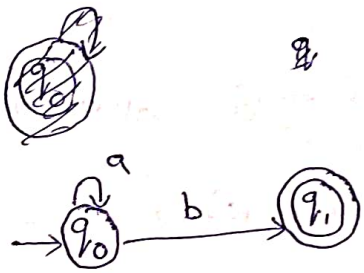
⑥ $R = (ab)^*$



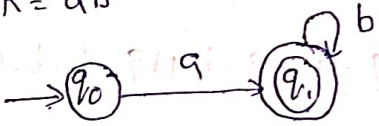
⑦ $R = a^*\,b^*$



⑧ $R = a^*b$



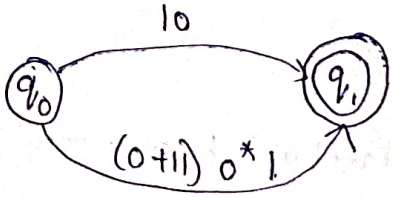
(i) $R = ab^*$



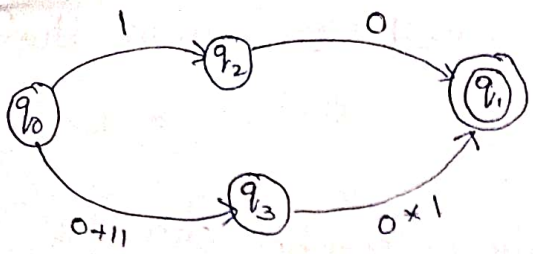
convert the following Regul Expression in to Finite autom

① $10+(0+11)\,0^*1$

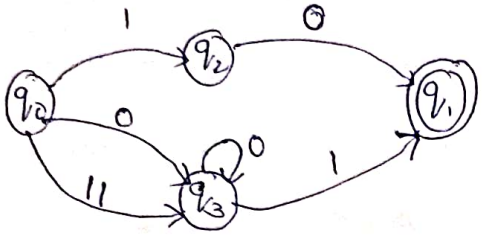
step ① : $R = a+b$

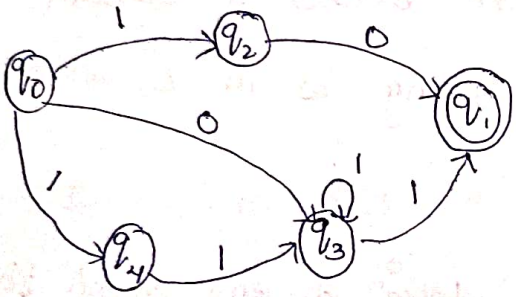


step ② :

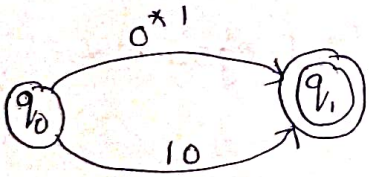


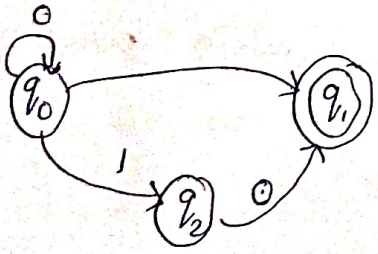
step ③ :



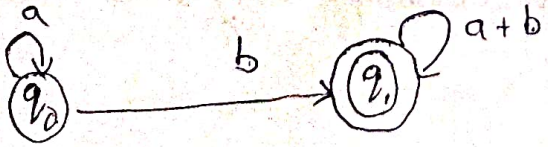


② $0^*1 + 10$

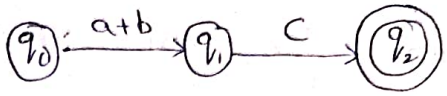


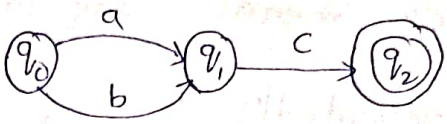


③ $a^*b\,(a+b)^*$

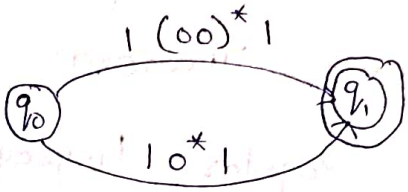


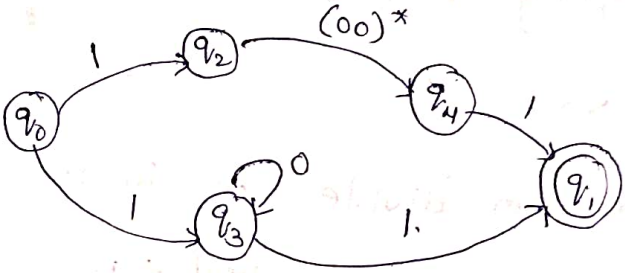
④ $(a+b)c$

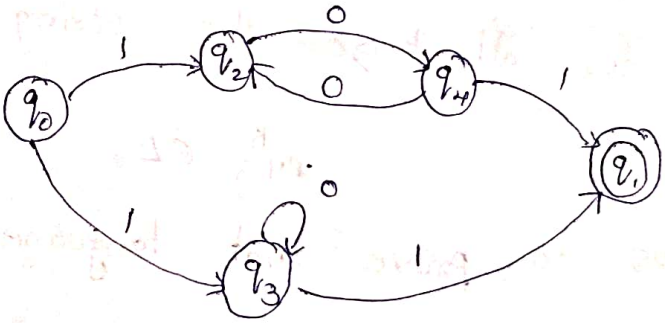




⑤ $1(00)^*1 + 10^*1$







Pumping Lemma: Pumping lemma is used to prove that a language is not regular language.

→ Pumping means any one of the input symbol can be Pushed repeatedly.

→ Lemma means proving.

Pumping Lemma Theorem:

→ If L is regular language, then there exists a constant 'n' (Pumping length) such that for every string ω in L where $|\omega| \geq n$.

→ we can divide ω in to 3 strings where $\omega = xyz$ such that

$|y| > 0$ $\quad$ $|xy| \leq n$

→ for all $k \geq 0$ the string $xy^k z$ is also belongs to L.

$$xy^k z \in L.$$

Steps to prove that language is not regular using pumping Lemma:

→ This is proved by contradiction method.

→ Assume L is regular.

→ Let n be the number of states in corresponding finite automa

→ choose a string ω such that $|\omega| \geq n$.

→ use pumping lemma to divide $\omega = xyz$ with $|xy| \leq n$, and $|y| > 0$

→ Find a suitable integer 'i' such that $xy^iz \notin L$, this contradicts our assumption so L is not regular.

① Prove that $L = \{a^p / p \text{ is Prime}\}$ is not a regular language.

$L = \{aa, aaa, aaaaa, \ldots\ldots\}$

$n = 3$, $\omega = aaa$ such that $|\omega| \geq n$.

Divide ω in to 3 parts as xyz

$x = a$ $y = a$ $z = a$. $|y| = |a| > 0$

$1 > 0$

$|xy| \leq n$.

$|aa| \leq 3 \Rightarrow 2 \leq 3$.

for $i \geq 0$, xyz is in. $\omega = xy^iz$

$i = 0 \rightarrow a\,a^0\,a \rightarrow aa \in L$

$i = 1 \rightarrow a\,a^1\,a \rightarrow aaa \in L$

$i = 2 \rightarrow a\,a^2\,a \rightarrow aaaa \notin L$

∴ L is not a regular language.

② Prove that $L = \{0^{n^2} / n \geq 0\}$ is not a regular language.

$L = \{\varepsilon, 0, 0000, 000000000, \ldots\ldots\}$

Let $n = 2$, $\omega = 0000$ such that $|\omega| \geq n \rightarrow 4 \geq 2$.

Divide ω in to 3 parts as $x\,y\,z$

$x = 0$ $y = 0$ $z = 00$ $|y| = |0| > 0 \Rightarrow 1 > 0$

$|xy| = |00| \leq 2 \Rightarrow 2 \leq 2$.

for $i \geq 0$, xyz is in $\omega = xy^iz$

$i = 0 \rightarrow 00^0 00 \rightarrow$ 0. ε. 00 → 000 ∉ L

∴ L is not a regular language.

③ Prove that $L = \{a^n b^n / n \geq 0\}$ is not regular language.

$L = \{\varepsilon, ab, aabb, aaabbb, \ldots\ldots\}$

$n = 2 \quad \omega = aabb$ such that $|\omega| \geq n$

$4 \geq 2.$

Divide ω in to 3 parts as $x\ y\ z$

$x = a \quad y = a \quad z = bb$ $\quad |y| > 0 \rightarrow |a| > 0 \rightarrow 1 > 0$

$|xy| \leq n \rightarrow |aa| \leq 2 \rightarrow 2 \leq 2.$

for $i \geq 0$, xyz in $\omega = x y^i z$

$i = 0 \longrightarrow \omega = a a^0 bb \longrightarrow a \cdot bb \notin L$

so that L is not a regular language.